

1 INTRODUCTION

1.1 Organization Profile

The Department of Chemistry at Madras Christian College (Autonomous) stands as a premier academic unit, profoundly dedicated to ensuring excellence in chemical education, scholarly inquiry, and advanced research. Established with a visionary mandate to foster scientific curiosity and rigorous analytical thinking, the department provides comprehensive training tailored across various specialized branches of chemistry, including Organic, Inorganic, and Physical Chemistry. The department prides itself on a rich legacy of emphasizing practical, hands-on learning, maintaining state-of-the-art physical laboratory facilities designed to cultivate the technical competencies of its students.

However, in response to modern educational paradigms and the increasing need for scalable, accessible learning methodologies, the department has recognized the critical importance of integrating sophisticated digital workflows into its traditional academic framework. The goal is not to replace the physical laboratory, but to augment physical learning with digital solutions that enhance student engagement, streamline complex administrative duties, and drastically increase the volume of practical exposure a student can experience. By embracing a digital transformation strategy, the department aims to dissolve traditional boundaries, transforming experimental practice from a localized, time-constrained activity into a continuous, data-driven academic journey.

1.2 Project Overview

The Virtual Chemistry Laboratory Management System (VCLMS) represents a paradigm shift in practical science education. It is an advanced, hybrid educational platform deliberately designed to bridge the gap between theoretical classroom learning and high-fidelity experimental execution. VCLMS meticulously integrates the robust administrative functionalities of a modern Learning Management System (LMS) with the real-time, interactive physics of a custom-built Virtual Laboratory

Simulation Engine.

Educational institutions historically encounter structural barriers that bottleneck effective practical learning. These include heavily restricted physical laboratory hours, the recurrent financial burden of purchasing volatile chemical reagents, and the inherent, non-trivial safety hazards associated with handling toxic or highly reactive substances. VCLMS confronts these systemic challenges by establishing a secure, persistent, and 24/7 accessible digital workspace. Here, students can engage in rigorous, repeatable scientific inquiry without the physical, temporal, or financial constraints that typically limit institutional capacities.

A central cornerstone of the VCLMS architecture is its technical evolution toward an entirely dynamic, data-driven environment. Moving away from the rigid, hardcoded simulations of the past, the project now features a groundbreaking "No-Code Experiment Builder." This allows chemistry faculty—regardless of software engineering expertise—to dynamically construct, configure, and modify complex multi-stage experimental procedures natively through the administrative UI. Faculty can define specific chemical endpoints, assign precise stoichiometric parameters, and configure validation sequences dynamically.

The simulation engine, powered by `p5.js`, directly parses these configurations from the robust PostgreSQL backend. It mathematically computes chemical states, rendering indicator-agnostic color morphology and realistic physical behaviors (such as the overfilling and realistic draining of volumetric apparatus). To maximize pedagogical rigor, the assessment phase strictly avoids automated hand-holding prompts; instead, students must initiate manual tracking of their sequential milestones. This requires genuine visual focus and technical competence, with the overarching system capturing granular procedural telemetry to determine the final analytical proficiency of the student.

Ultimately, this unified approach ensures that theoretical knowledge is substantially reinforced through risk-free, highly analytical practical applications, fostering a comprehensive mastery of chemical principles within a sustainable, scalable digital ecosystem.

1.3 Existing System

The existing infrastructural and administrative frameworks in many traditional chemistry departments remain heavily reliant on manual, analog paradigms. Management of student performance, daily lab attendance, and serialized academic results frequently relies on paper-based record-keeping or fragmented spreadsheet data. Instructors are burdened with the daily maintenance of physical registers, a tedious process inherently vulnerable to clerical errors, accidental data loss, and severe delays in reporting. This decentralized method forces student records into non-searchable, physical formats, thereby obscuring critical trends in long-term academic progress and preventing faculty from executing timely educational interventions. Furthermore, communicating finalized results or academic updates often depends on physical notice boards, creating disjointed feedback loops.

Beyond administrative inefficiencies, traditional practical science education is fundamentally constrained by physical infrastructure. Due to the significant recurrent costs of consumable reagents and stringent occupational safety requirements attached to handling concentrated acids or glassware, academic institutions must severely curtail student laboratory access strictly to supervised hours. This physical scarcity implicitly prevents continuous, self-paced student practice. Consequently, practical sessions are frequently compromised; experiments are forced into large-group dynamics where some students are relegated to the role of passive observers rather than active participants. Such a restrictive environment hinders the individual acquisition of nuanced technical skills like precise burette manipulation or accurate endpoint observation.

In terms of existing digital alternatives, legacy virtual laboratories frequently suffer from monolithic, hardcoded designs. They operate as static animations rather than reactive physical simulations, pre-programmed for very specific, unalterable scenarios. Faculty cannot easily generate custom laboratory variations for targeted assessments without triggering massive software redevelopment overheads. Furthermore, these older platforms critically lack an intelligent, continuous evaluative layer; they fail to track the granular procedural mistakes a student makes during an experiment, relying instead on evaluating only the final inputted numerical answer.

1.4 Proposed System

To directly counteract the pedagogical and administrative deficiencies of traditional models, VCLMS introduces a cohesive, hybrid digital framework. By centralizing the entirety of fundamental academic activities into a single, secure Django-based portal, the proposed system entirely eliminates the inefficiencies of manual record-keeping while granting students unrestricted, asynchronous access to crucial laboratory practice.

The defining hallmark of the proposed system is its deployment of a computationally intensive, 100% No-Code Experiment Platform. This highly innovative module transfers total authoring control from software developers directly to the academic faculty. Through a series of intuitive database-backed configurations, dynamic property dropdowns, and robust chemical selection modals, educators can dynamically build intricate procedures, such as Double Indicator Titrations, entirely from the user interface.

When a student executes these configured labs, the p5.js simulation engine dynamically retrieves the JSON-structured parameters and injects them into the real-time simulation canvas. The engine executes sophisticated mathematical calculations to simulate precise, indicator-agnostic color morphing and continuous volumetric physics. Crucially, the proposed system enforces a rigorous assessment rubric. It utilizes a manual, student-initiated observation pipeline that actively tracks sequential milestones based on the exact cataloged chemical names in the database. The environment permits realistic technical errors—such as fluid overflow, improper apparatus alignment, and inaccurate titration rates—logging these infractions within a continuous "Marking Manager" engine.

Upon completion, the system executes mathematical verifications of the student's inputs against the hidden, dynamically generated endpoints, executing complex point penalties and ultimately generating an authenticated, secure PDF Appraisal Report synchronized into the PostgreSQL database.

1.5 System Configuration

1.5.1 Hardware Configuration

The VCLMS system requires a robust computational environment, particularly on the client side, to seamlessly process dynamic geometric rendering, real-time physics tracking, and simultaneous server communications without latency.

- Processor: Intel Core i5 or higher (Recommended for handling p5.js canvas rendering and Django server concurrently).
- RAM: Minimum 8 GB (Ensures smooth performance for the interactive laboratory simulation).
- Storage: 500 MB of available disk space for local application files and the Postgre SQL database.
- Network: An active internet connection is required for accessing cloud-hosted materials and receiving Firebase-based notifications.

1.5.2 Software Configuration

The technical stack of the VCLMS project leverages Python's computational power and modern web technologies for a seamless user experience.

- Operating System: Windows 10/11, Linux, or macOS.
- Front-End: HTML5, CSS3, JavaScript (ES6+ core logic execution), p5.js (The primary 2D Physics and Canvas-rendering Simulation engine), and Bootstrap 4 for responsive dashboard architectures.
- Back-End: Python 3.9+ (Core programming logic).
- Web Framework: Django 4.0+ (Robust MVT architecture for the Management System).
- Database: SQLite (Default for development) or PostgreSQL (Recommended for production environments).
- Development Tools: Visual Studio Code (IDE), Git (Version Control).

1.5 Objectives of the Project

The primary goal of VCLMS is to revolutionize traditional science education through digital transformation. The specific objectives are:

- **Virtualize Practical Science Learning:** To develop a high-fidelity Virtual Chemistry Laboratory where students can perform Titration experiments with real-time feedback.
- **Pioneer Code-Free Experiment Design:** To strictly remove technical barriers for educators by implementing a comprehensive No-Code Experiment Builder. This allows chemistry faculty to autonomously build, parameterize, and deploy complex experimental simulations dynamically via the UI over a PostgreSQL database schema.
- **Integrate Administrative Management:** To design a centralized Learning Management System (LMS) that automates enrollment, course scheduling, and academic record tracking.
- **Implement Procedural Evaluation:** To advance beyond standard input-based grading by designing an intelligent "Marking Manager" logic engine. This system continuously captures procedural telemetry, evaluates precise operational techniques, and applies a rigid automated penalty system enforcing professional lab discipline.
- **Streamline Academic Analytics:** To automate the generation of dynamic PDF lab reports and dashboard analytics for real-time tracking of attendance and overall performance.

2 BACKGROUND STUDY

2.1 Integrating Virtual Environments in Learning Management Systems

The integration of specialized virtual laboratory environments within broader Learning Management Systems (LMS) marks a profound, systemic advancement in the delivery of science education. Historically, an LMS was strictly utilized as a static digital repository for document distribution, generalized announcements, and rudimentary grade tracking. However, the modern pedagogical requirement for "active" and "experiential" learning has necessitated the transformation of these platforms into dynamic hubs for highly interactive content.

By strategically hosting high-fidelity laboratory simulations natively within a managed LMS framework, academic institutions can establish a highly structured, continuous learning path. This architecture effectively bridges the historically fragmented gap between theoretical reading (e.g., watching a lecture on titration) and practical application (e.g., physically executing a titration protocol). A centralized digital ecosystem ensures that granular experimental telemetry, daily student progress, and complex academic records are continuously synchronized. This provides faculty with a comprehensive, holistic, and uninterrupted view of a student's technical development, vastly superior to the disjointed academic tracking inherent in traditional, offline educational methods. Furthermore, executing this within a unified Django portal ensures secure Role-Based Access Control (RBAC), meaning the transition from the theoretical study dashboard to the secure experimental simulation is protected and tracked definitively by user identity.

2.2 Pedagogical Benefits and Cognitive Load in Virtual Labs

The foundational mechanics of Virtual Laboratory Simulations (VLS) are deeply grounded in the established theory of "experiential learning," which posits that genuine, profound understanding is achieved specifically through an iterative cycle of action, observation, and critical reflection. A significant barrier to experiential

learning in physical chemistry laboratories is the concept of "extraneous cognitive load." In a traditional lab, students are burdened heavily with physical safety anxieties—fear of dropping fragile glassware, the catastrophic risk of spilling concentrated acids, or the pressure of wasting expensive chemical reagents. This extraneous cognitive overhead severely distracts from the "intrinsic" cognitive load required to actually learn complex chemical principles, such as calculating molarity or observing stoichiometric transitions.

In a fully virtualized lab, students are granted the psychological and procedural freedom to make mistakes and repeatedly explore "what-if" scenarios entirely free from physical risks. This safety net deliberately minimizes safety-related cognitive load, redirecting the student's entire mental focus towards the underlying logic of the experimental procedure. To ensure this virtual environment does not dilute pedagogical challenge, VCLMS explicitly removed previously implemented automatic on-screen assessment prompts. Instead of the software automatically detecting and announcing an endpoint, the student is forced into a "student-initiated manual assessment" workflow. This ensures that the transition to a virtual space enhances, rather than diminishes, academic rigor and observational accountability.

2.3 The Role of Real-Time Interactivity and Feedback

Interactivity is the defining characteristic of a successful virtual laboratory, transforming passive observation into active scientific inquiry. Sophisticated simulation engines use a "State-Driven" approach where every student interaction such as adjusting a burette tap or swirling a flask results in immediate visual and logical changes in the virtual workspace. This real-time feedback loop is essential for teaching students the discipline of the laboratory, as it allows them to immediately see the consequences of their actions, such as a localized color change during titration. In VCLMS, this interactivity is further enhanced by an intelligent marking layer that monitors student performance at a granular level, ensuring that the simulation acts not just as a visual tool but as a digital tutor that guides the student toward professional laboratory standards.

2.4 Technical Standards for Modern Web-Based Simulations

The technical development of virtual labs has shifted toward standardized web technologies like HTML5 and JavaScript libraries such as p5.js, which allow for high-performance rendering without external plugins. These frameworks enable developers to create responsive, hardware-accelerated canvases that simulate physical properties like the meniscus level of a liquid or gradual color transitions using precise mathematical interpolation. By utilizing a canvas-based approach, VCLMS can render smooth animations and handle complex event listeners for drag-and-drop interactions, ensuring that the simulation feels fluid and intuitive to the user. This reliance on modern web standards guarantees that the Virtual Chemistry Laboratory is accessible across a wide range of devices, providing a scalable solution for institutions looking to deliver high-quality practical science education to a global student base.

2.5 Evolution of Authoring Tools in Educational Simulations (The No-Code Paradigm)

A highly profound limitation of first-generation educational software was its absolute rigidity; experiments were hardcoded comprehensively into the application's source code. If an academic institution wished to alter a titration endpoint, introduce a new chemical indicator, or modify the penalty parameters for an assessment, they were fundamentally reliant on commissioning dedicated software developers. This created an unsustainable bottleneck, vastly reducing the long-term viability and adaptability of the software as standard chemistry curriculums evolved.

To circumvent this monumental flaw, VCLMS pioneered the integration of a “No-Code Experiment Configuration Platform”. This paradigm essentially democratizes software authoring, passing dynamic architectural control directly into the hands of academic educators and Department Heads. Utilizing advanced administrative dashboards, faculty are empowered to construct brand-new, complex multi-stage experimental protocols natively through the browser.

This is achieved utilizing an intricate material-sync architecture, relying on dynamic

property dropdowns, complex chemical selection modals, and precise sequencing logics. The finalized educator inputs are rapidly serialized into extensive, highly structured JSON payloads and pushed securely to the centralized PostgreSQL database. Because the underlying p5.js simulation engine is uniquely built to parse dynamic metadata rather than relying on hardcoded scripts, it can physically manifest these teacher-configured parameters instantly. This breakthrough not only severs the dependency on continuous software engineering interventions but definitively ensures the platform functions as an infinitely scalable, permanently adaptable hub for scientific inquiry.

3 SYSTEM DESIGN

3.1 Module Descriptions

3.1.1 Authentication & Role-Based Access Control (RBAC) Module

The Authentication Module is the primary security layer of the VCLMS, responsible for managing user identities, session persistence, and role-based access levels. This module allows for the creation of distinct user profiles for HODs (Admin), Faculty (Staff), and Students, ensuring that each user type is granted only the permissions necessary for their academic role. By using Django's secure authentication framework, student credentials and profile data are stored with high-level encryption and CSRF protection to prevent unauthorized data access. Furthermore, the module maintains a persistent session for each user, allowing them to navigate between lesson materials and the virtual lab while consistently tracking their individual progress and administrative history within the centralized database.

3.1.2 No-Code Experiment Configuration Module

A revolutionary pillar of the system's administrative design is the No-Code Experiment Configuration Module. Historically, digital simulation parameters were strictly hardcoded, meaning modifying a chemical endpoint or validation sequence required extensive developer reprogramming. This module fundamentally decentralizes authoring power, transferring complex simulation logic entirely via a graphical User Interface (UI) into the hands of chemistry faculty.

This highly advanced module operates through a deeply structured dynamic property architecture. Faculty interact with specialized, database-driven dropdowns and robust chemical selection modals to define critical parameters. The most sophisticated feature is its "Material Sync" logic; when an educator selects a chemical from the drop-down catalog, the module autonomously synchronizes its exact molecular nomenclature and hidden mathematical thresholds directly into the configuration payload.

Once the educator finalizing the setup, the module serializes the entire protocol into a highly dense JSON structure, transmitting and securing it perpetually within the PostgreSQL backend. This completely bypasses the need for manual programming while ensuring unlimited scalability for designing countless chemical variations through one unified logic hub.

3.1.3 Virtual Laboratory Simulation Module

The Virtual Laboratory Module represents the technical core of the system, providing an interactive, state-driven environment for performing complex chemistry experiments like Double Indicator Titration. This module integrates the p5.js simulation engine with a dedicated "Marking Manager" logic layer to monitor student behavior and procedural accuracy in real-time. Students interact with digitized laboratory apparatus to perform volumetric analysis, while the system provides immediate visual feedback through realistic color changes and meniscus level calculations. Beyond mere visualization, this module records all experimental observations such as the V1 and V2 endpoints and automatically verifies the student's mass calculations, ensuring that practical proficiency is evaluated with mathematical precision.

3.1.4 Academic Management (LMS) Module

The Academic Management Module serves as the administrative backbone of the platform, organizing the delivery of study materials, video courses, and academic records. This module allows faculty to upload subject-specific content, manage academic sessions, and track student attendance through a unified dashboard interface. It automates the calculation of internal assessment marks by correlating lab performance with overall course progress, thereby reducing the manual documentation burden for the department staff. Additionally, this module is responsible for the automated generation of authenticated PDF lab reports and progress certificates, providing students with a tangible record of their performance that is archived in the system for future reference and administrative audit.

3.2 Workflow of The Proposed System

The workflow of the VCLMS is designed to provide a seamless and logical progression for the student, starting from the initial registration in the department portal to the final generation of an experiment appraisal. Upon secure login, the student enters the Learning Management dashboard, where they can access theoretical study materials and video lectures to build foundational knowledge before attempting practical work. Once prepared, the student transitions to the Virtual Laboratory Module, where they perform the titration experiment under the supervision of the digital marking engine. The overall workflow ensures that the platform is efficient and student-centric, guiding users through every stage of the academic cycle from practice and execution to receiving automated grades as illustrated in the System Workflow Diagram (Fig 3.1).

3.2.1 Laboratory Execution Sub-module

Within the execution sub-module, the system demands exhaustive physical discipline via the interactive canvas. The student enters the setup phase, where they must precisely align their digital apparatus—dragging, dropping, and validating the burette placement, the funnel insertion, and the analyte formulation.

The primary innovation during the titration phase revolves around the forced "Student-Initiated Manual Assessment" workflow. Rather than holding the student's hand—whereby the system automatically pauses the experiment and signals that a chemical endpoint has materialized—the student must exhibit profound visual acuity. They must manually govern the titration flow rate, aggressively observe the continuous stoichiometric color shift, halt the procedure autonomously upon spotting the precise endpoint, and manually trigger the observation save.

Concurrently running silently in the background is the overarching Marking Manager. This continuous telemetric proctor monitors every micro-interaction. If a student forgets to zero an initial burette reading, excessively overfills a chemistry

vessel due to negligence, or attempts a titration without maintaining required flask agitation, the Marking Manager immediately logs explicit procedural faults. The simulation explicitly utilizes exact chemical name validation against the catalog, severely penalizing random guesses or technically inaccurate setups.

3.2.2 Calculation & Results Sub-module

Following the practical simulation, the student enters the calculation sub-module, where they must input their observed volumes and perform molarity-based mass calculations for the given samples. The system accepts user input for variables and uses a background math engine to verify the accuracy of the student's final results within a predefined tolerance level. If the calculations are correct, the system marks the milestone as complete; if errors are detected, the system logs specific penalties and provides remedial feedback. Finally, the sub-module aggregates all session marks and penalties to generate a comprehensive lab report, completing the academic cycle for that specific experiment.

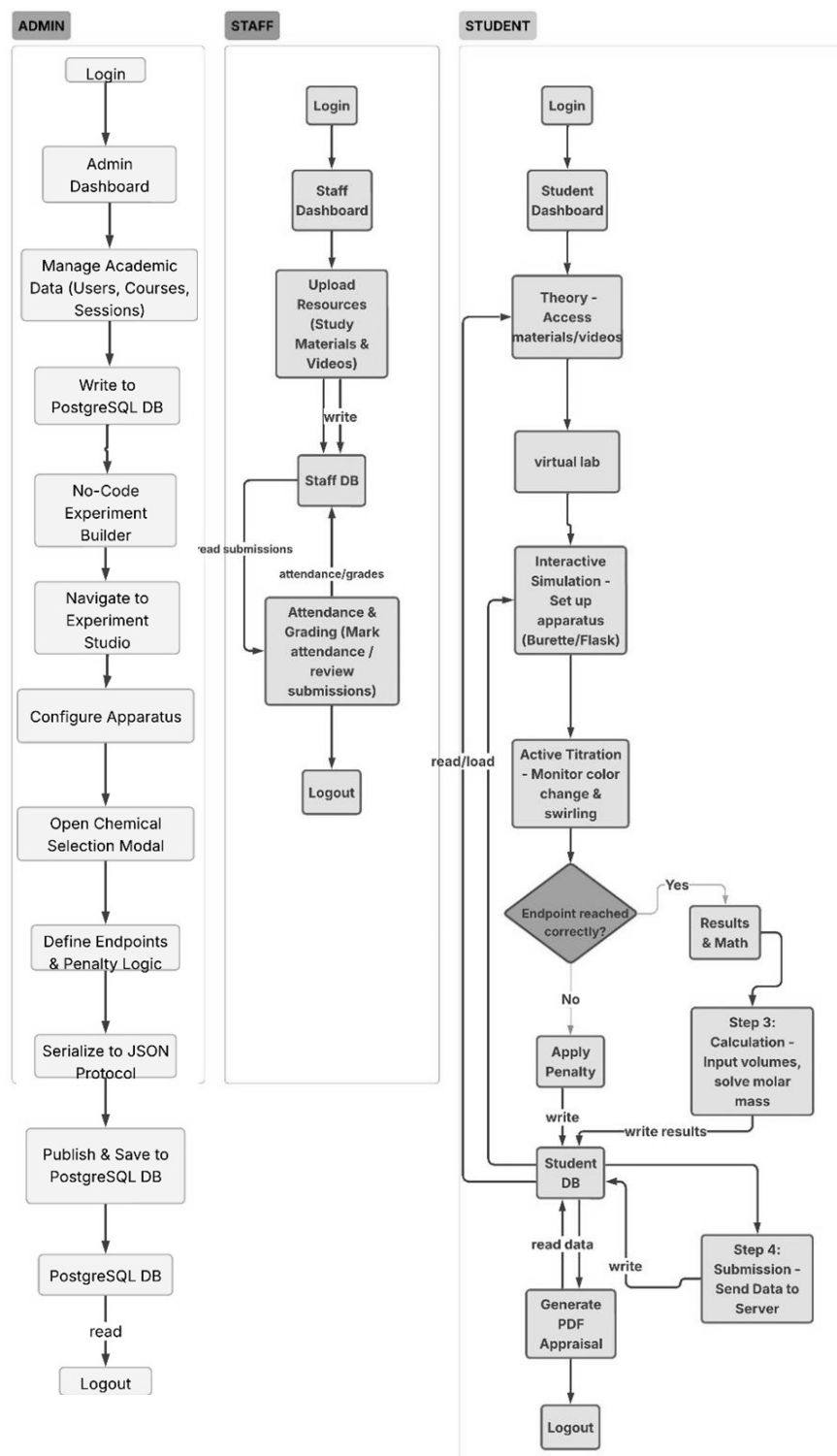


Fig. 3.1 System Workflow Diagram

4 DATABASE DESIGN

The database design for the Virtual Chemistry Laboratory Management System (VCLMS) is structured to efficiently manage user authentication, session control, and complex experimental data. PostgreSQL is used as the primary relational database system due to its robust performance, scalability, and support for complex queries, making it highly suitable for a high-concurrency academic platform. The database follows a relational model where the CustomUser table acts as the primary entity for identity management, while other tables like VirtualLabSubmission and Attendance are linked through relational keys to ensure data consistency. This design choice ensure seamless retrieval of academic records, accurate tracking of student laboratory progress, and secure storage of administrative configurations in a production-ready environment.

4.1 Users Table

Stores user registration details such as email, name, and encrypted password for secure authentication and role-based login management across the HOD, Staff, and Student profiles.

Table 4.1. User Table

Column Name	Data Type	Constraints
id	int	Primary Key Auto Increment
email	varchar(254)	Required, Unique
first_name	varchar(150)	Required
last_name	varchar(150)	Required
user_type	varchar(1)	Required (1: Admin, 2: Staff, 3: Student)
gender	varchar(1)	Required (M/F)
created_at	datetime	Default: Current timestamp

4.2 Experiment Configuration Table

This is the newly architects pillar of the database that singularly facilitates the No-Code Experiment Configuration Module. Rather than storing rigid chemical variables across dozens of tiny columns, this table utilizes a flexible JSON schema to store entire hierarchical protocols authored by the faculty.

Table 4.2. Experiment Configuration Table

Column Name	Data Type	Constraints
id	int	Primary Key Auto Increment
faculty_id	int	Foreign Key (References `admin.id`)
experiment_title	varchar(255)	Required
apparatus_config	jsonb	Required
chemical_logic	jsonb	Required
created_at	datetime	Default: Current timestamp

4.3 Virtual Lab Submission Table

The Submission table operates as the secure vault for the "Marking Manager's" telemetry. Whenever a student completes a manual experiment, this table is vigorously updated. It proves that the student successfully navigated the simulation, capturing not merely the final score but the exact mathematical inputs and granular procedural faults they incurred.

Table 4.3. Virtual Lab Submission Table

Column Name	Data Type	Constraints
id	int	Primary Key Auto Increment
student_id	int	Foreign Key (References Student.id)
experiment_name	varchar(200)	Required
v1_observed	float	Required
v2_observed	float	Required
calc_na2co3	float	Required
calc_nahco3	float	Required
total_score	int	Required
penalty_log	text	Required

4.4 Attendance Table

Maintains daily participation data for students across various subjects and academic sessions, ensuring proper tracking for internal assessment eligibility and records.

Table 4.4. Attendance Table

Column Name	Data Type	Constraints
id	int	Primary Key Auto Increment
session_id	int	Foreign Key (References Session.id)
subject_id	int	Foreign Key (References Subject.id)
date	date	Required
created_at	datetime	Default: Current timestamp

5 MODULE DESIGN

5.1 Authentication and Identity Management Module

The Authentication Module is the primary security gateway for the VCLMS, ensuring that all interactions within the platform are attributed to specific, verified user profiles. It implements a sophisticated Role-Based Access Control (RBAC) mechanism that dynamically allocates permissions based on the user's designation as an HOD, Faculty Staff, or Student. This module utilizes Django's secure hashing algorithms to store student credentials, protecting the integrity of academic records and experimental results from unauthorized access or manipulation. Beyond mere login management, the module maintains persistent session states that allow students to transition seamlessly between theoretical study materials and the high-stakes virtual laboratory environment without losing their progress. By centralizing the identity layer, the system ensures that every attendance record, quiz score, and lab appraisal report is accurately linked to the correct individual for institutional auditing.

5.2 Virtual Laboratory Module (The Simulation Core)

5.2.1 Real-Time Simulation Engine

The Simulation Engine represents the technical center of gravity for the project, utilizing the p5.js library to render a physics-compliant, interactive laboratory workspace. This sub-module manages a complex coordinate-based canvas where various glassware and apparatus are treated as intelligent objects with unique chemical and volumetric properties. It handles the continuous rendering of the meniscus level, the realistic flow rate of liquids from the burette, and the dynamic color transitions of the titration mixture based on real-time pH calculations. The engine's 60-FPS refresh rate provides a fluid experience that mimics the physical reality of a chemistry lab, allowing students to develop the visual precision necessary for accurately reading volumetric scales and identifying chemical endpoints.

5.2.2 Apparatus Setup and Handling Logic

The Setup and Handling sub-module is responsible for enforcing the precise procedural standards required for professional laboratory work during the initial stages of an experiment. It monitors every student interaction with the apparatus catalog, ensuring that equipment like the burette and conical flask are properly cleaned and positioned before any chemical reagents are introduced. This module implements specific logic-checks, such as verifying if the student zeroed the burette reading or if they removed the funnel before beginning the titration process, with immediate penalties applied for any technical oversights. By providing a safe, repeatable environment for apparatus manipulation, this sub-module allows students to build the "muscle memory" and procedural discipline required for conducting physical experiments without the risk of reagent wastage or equipment breakage.

5.2.3 Titration Execution and Observation

The Execution and Observation sub-module manages the interactive heart of the titration experiment, where students must balance visual observation with controlled digital manipulation. It simulates the delicate process of adding HCl to the analyte mixture, requiring the student to maintain constant agitation through a "Swirl" mechanic while simultaneously monitoring the burette flow. The system specifically models the two-stage endpoint of a double-indicator titration, triggering realistic color shifts—from pink to colorless and then to red—based on the simulated volumetric thresholds. Students must use their judgment to stop the flow at the exact moment of color change and record their observations, simulating the high-pressure decision-making process inherent in actual volumetric analysis.

5.2.4 Marking Manager and Penalty Logic

The Marking Manager acts as the automated evaluative intelligence of the lab, continuously comparing student actions against a standardized digital rubric of laboratory excellence. This sub-module processes granular behavioral data, such as titration speed and the regularity of flask agitation, to determine if the student

followed the most efficient and pedagogically sound experimental path. It applies a series of point deductions for "procedural neglect," such as titrating too fast or adding indicators in the wrong sequence, ensuring that students are rewarded for their technique rather than just the accuracy of their final result. This real-time scoring system provides students with immediate feedback on their proficiency, transforming the simulation from a simple visualization tool into an advanced, objective platform for practical skills assessment.

5.3 Academic Content and Learning Module

The Academic Content Module provides the theoretical foundation of the VCLMS, organizing a curated library of instructional resources designed to prepare students for their practical laboratory work. It facilitates the upload and logical categorization of subject-specific study materials, including deep-dive PDF guides and professional video lectures that cover the foundational chemistry principles behind volumetric analysis. This module allows faculty to manage instructional sessions and track student engagement with the materials, ensuring that theoretical concepts are mastered before the student attempts the laboratory simulation. By integrating these theoretical resources directly into the management platform, the system ensures a holistic educational cycle where reading and watching are directly reinforced by the subsequent practical execution in the virtual lab.

5.4 Assessment and Mathematical Verification Module

The Assessment Module handles the quantitative evaluation of the student's mastery, focusing on both theoretical knowledge through automated quizzes and mathematical accuracy in experimental calculations. Once a titration attempt is completed, this module challenges the student to input their recorded observations and solve molarity-based mass-equivalence formulas to derive the final sample weights. The system's math engine retrieves the true values from the hidden simulation state and compares them against the student's calculations using a strict 0.05g tolerance window. This rigorous verification process ensures that students not only perform the laboratory actions correctly but also possess the mathematical proficiency required to translate their practical observations into scientifically valid experimental findings.

5.5 No-Code Experiment Builder Module

Functioning as a high-end authoring architecture, this module replaces antiquated hardcoded text inputs entirely with a suite of dynamic UI configurations. This module allows an educator to sequentially build an entire simulation. By utilizing the *Chemical Selection Modal*, the faculty selects specific reactivities precisely matched to the backend database catalog. The module then processes these selections through a "Material Sync" pipeline, binding dynamic properties (like specific gravity, molarity, indicator limits, and required procedural milestones) into a unified JSON protocol. When published, this structured data acts as the absolute blueprint that the `p5.js` engine natively executes on the client-side browser, ensuring infinite experimental scalability without requiring software source code modification.

6 CODING

lab_engine.js:

```
let imgLabBg;
let vessels = {};
let isDragging = null;
let hoverVessel = null;

let studentVolume = 0;
let phStage = 0;
let beakerTargetVol = 0;
let pipetteTargetVol = 0;
let buretteTargetVol = 0;
let userClosedZoom = false;
let buretteReading = 0;

let apparatusCatalog, chemicalCatalog;
let currentCatalogTab = 'apparatus';
let imgBeaker, imgBottle, imgBurette, imgPipette;
let imgConical, imgVolumetric, imgFunnel, imgWash, imgBunsen;
let imgBalance, imgCrucible, imgHotplate, imgLiebig, imgMeltingPoint, imgPHMeter,
imgSepFunnel, imgTLC;
let imgBuretteTube, imgStand;

let catalogVisible = false;
let catalogToggleButton = null;
let catalogPanelBounds = null;
let controlsVisible = false;
let assistantVisible = true;
let clearShelfButton = null;

let currentPositions = null;
let idCounter = 0;
const sizeMultiplier = 0.45; // Slightly reduced for better fit on shelves
let labSurfaces = null;
const BURETTE_GLASS_X_OFFSET = -8;
// Initialize from Django (passed via template)
// Try finding the global variable first, then fallback to the script tag (json_script)
let experimentData = typeof EXPERIMENT_CONFIG !== 'undefined' ?
EXPERIMENT_CONFIG : null;
if (!experimentData) {
  const configEl = document.getElementById('experiment-config-data');
  if (configEl) {
    try {
      experimentData = JSON.parse(configEl.textContent);
      console.log("Config loaded from script element.");
    } catch (e) {
      console.error("Failed to parse experiment-config-data element:", e);
    }
  }
}
```

```

    }
  }
}

const TARGET_V1 = experimentData?.targets?.v1 || 10.0;
const TARGET_V2 = experimentData?.targets?.v2 || 25.0;

// Wire up the dynamic catalogs so getApparatusProps() works
window.APPARATUS_CATALOG = experimentData?.catalogs?.apparatus || [];
window.CHEMICAL_CATALOG = experimentData?.catalogs?.chemicals || [];
window.REACTION_CATALOG = experimentData?.catalogs?.reactions || [];

let currentStepIndex = 0; // Tracks which milestone we are on
let penalties = []; // List of strings explaining point losses
let sessionMarks = 0; // Current score
let activeModal = null; // Tracks if an input box is open

const LAB_SURFACES = {
  shelfTop: { y: 85, minX: 280, maxX: 950, shadowAlpha: 30 },
  shelfBottom: { y: 175, minX: 280, maxX: 950, shadowAlpha: 30 },
  table: { y: 335, minX: 50, maxX: 1150, shadowAlpha: 45 }
};

//experiment assessment
class MarkingManager {
  constructor(config) {
    this.config = config;
    this.milestones = config?.milestones || [];
    this.completedIds = new Set();
    this.mistakesMade = new Set();
    this.buretteProperlyZeroed = false;

    // Stored observations for the calculation phase
    this.recordedV1 = 0;
    this.recordedV2 = 0;
    this.swirlNeglectTimer = 0; // Track lack of swirling
  }

  // This MUST be called inside the p5.js draw() loop
  update() {
    const flask = Object.values(vessels).find(v => v.type === 'conical_flask');
    const burette = Object.values(vessels).find(v => (v.type === 'burette' || (v.type === 'burette_tube' && v.mountedTo)));

    // GENERIC BURETTE PENALTIES (Apply to all titration experiments)
    if (burette && (this.config?.type === 'double_indicator' || this.config?.type === 'simple_titration')) {
      let reading = abs(burette.capacity - burette.targetVolume);

      if (reading <= 0.3) {

```

```

    this.buretteProperlyZeroed = true;
  }

  // Penalty: Titrating without Zeroing
  if (keyIsDown(32) && flask) {
    const snapX = burette.type === 'burette' ? (burette.x +
BURETTE_GLASS_X_OFFSET) : burette.x;
    const dripTipY = burette.type === 'burette' ? (burette.y + 120) : (burette.y +
burette.h * 0.4);

    if (dist(flask.x, flask.y, snapX, dripTipY) < 100) {
      if (!this.buretteProperlyZeroed && reading > 0.3) {
        this.addPenalty("no_zeroing", 15, "Titrating without zeroing the burette first.");
      }
    }
  }

  // Penalty: Titrating without removing funnel
  if (keyIsDown(32) && !burette.hasFunnel) {
    this.addPenalty("no_funnel", 5, "Started titration without using a funnel (or funnel is
missing).");
  }

  // Penalty: Titrating without Swirling (Lack of Agitation)
  if (keyIsDown(32) && !keyIsDown(87)) {
    this.swirlNeglectTimer++;
    if (this.swirlNeglectTimer > 180) { // ~3 seconds of neglect
      this.addPenalty("no_swirl", 10, "Titrating without swirling the flask regularly.");
    }
  } else {
    if (keyIsDown(87)) this.swirlNeglectTimer = 0;
  }
}

let m = this.milestones[currentStepIndex];
if (m && !this.completedIds.has(m.id)) {
  let allRulesPassed = true;
  this.currentStepHint = ""; // Reset hint for current frame

  if (m.rules && m.rules.length > 0) {
    for (let rule of m.rules) {
      let targetVessel = Object.values(vessels).find(v =>
        v.type === rule.target_vessel || v.chem === rule.target_vessel ||
        (rule.target_vessel === "burette" && (v.type === "burette_tube" || v.type ===
"burette"))) ||
        (rule.target_vessel === "burette_tube" && (v.type === "burette_tube" || v.type
=== "burette")))
      );

      if (!targetVessel) {

```

```

    allRulesPassed = false;
    this.currentStepHint = `X Required apparatus (${rule.target_vessel}) not found.
Check blueprint naming.`;
    break;
}

// Extract the property value to check generically
let propValue = 0;
if (rule.target_property === "reading") {
    propValue = targetVessel.capacity - targetVessel.targetVolume;
} else if (rule.target_property === "volume") {
    propValue = targetVessel.targetVolume;
} else if (rule.target_property === "pH") {
    propValue = targetVessel.contents ? (targetVessel.contents.pH || 7.0) : 7.0;
} else if (targetVessel.contents) {
    // Check if property is a chemical name
    const chemRef = targetVessel.contents.chemicals ?
targetVessel.contents.chemicals[rule.target_property] : null;
    if (chemRef) {
        propValue = chemRef.volume;
    } else if (targetVessel.contents.indicators &&
targetVessel.contents.indicators[rule.target_property] !== undefined) {
        propValue = targetVessel.contents.indicators[rule.target_property];
    } else {
        // If we are looking for a chemical that ISN'T there, propValue is 0.
        propValue = 0;
        if (rule.operator !== "<" && rule.operator !== "<=") {
            this.currentStepHint = `Missing chemical: ${rule.target_property} in
${rule.target_vessel}.`;
        }
    }
}

// Evaluate generic operator string
let rulePassed = false;
switch (rule.operator) {
    case ">=": rulePassed = (propValue >= rule.value); break;
    case ">": rulePassed = (propValue > rule.value); break;
    case "<=": rulePassed = (propValue <= rule.value); break;
    case "<": rulePassed = (propValue < rule.value); break;
    case "==": rulePassed = (abs(propValue - rule.value) < 0.1); break; // 0.1 tolerance
for vol
    case "!=": rulePassed = (abs(propValue - rule.value) > 0.1); break;
    case "CONTAINS": rulePassed = (propValue > 0); break;
}

if (!rulePassed) {
    allRulesPassed = false;
    if (propValue === 0 && (rule.operator === ">=" || rule.operator === "==" ||
rule.operator === "CONTAINS")) {

```

```

        this.currentStepHint = `Missing chemical: ${rule.target_property} in
        ${rule.target_vessel}`;
      } else {
        let opLabel = rule.operator === "==" ? "exactly" : (rule.operator === ">=" ? "at
least" : rule.operator);
        this.currentStepHint = `${rule.target_property} should be ${opLabel}
        ${rule.value}`;
      }
      break;
    }
  } else {
    allRulesPassed = true;
  }

  // --- NEW ASSESSMENT LOGIC ---
  const hasPrompts = (m.observation_prompts && m.observation_prompts.length > 0)
  ||
    (m.calculation_prompts && m.calculation_prompts.length > 0);

  if (hasPrompts) {
    // Rule-based validation is now just a state for the Manual Button
    this.rulesValidated = allRulesPassed;
    if (!allRulesPassed && this.currentStepHint) {
      // Show hint in the drawer, but don't auto-complete
    }
  } else {
    // For non-prompt steps (like Fill Burette), keep automatic completion
    if (allRulesPassed) {
      this.currentStepHint = "";
      this.completeMilestone(m.id);
    }
  }
}

handleManualSubmit() {
  let m = this.milestones[currentStepIndex];
  if (!m) return;

  // RULE PENALTY: If student submits before validation rules are met (e.g. premature
  endpoint)
  if (!this.rulesValidated) {
    this.addPenalty("premature_submit_" + m.id, 15, `Premature assessment: Rules for
    ${m.desc} were not met.`);
  }

  // Proceed with assessment prompts regardless of penalty
  this.completeMilestone(m.id);
}

```

```

completeMilestone(id) {
  if (this.completedIds.has(id)) return;
  let m = this.milestones.find(item => item.id === id);
  if (!m) return;

  // Trigger Generic Observation Prompts Sequentially
  if (m.observation_prompts && m.observation_prompts.length > 0) {
    for (let p of m.observation_prompts) {
      let val = window.prompt(`LAB OBSERVATION:\n${p.description}`);
      if (val !== null && val.trim() !== "") {
        let userVal = parseFloat(val);
        let trueVal = 0;
        if (p.title.toLowerCase() === 'v1' && experimentData?.targets?.v1) {
          trueVal = experimentData.targets.v1;
        } else if (p.title.toLowerCase() === 'v2' && experimentData?.targets?.v2) {
          trueVal = experimentData.targets.v2;
        } else {
          let targetVessel = Object.values(vessels).find(v => v.type === p.target_vessel ||
            (p.target_vessel === "burette" && (v.type === "burette" || (v.type === "burette_tube"
              && v.mountedTo))));
          if (targetVessel) {
            if (p.target_property === "reading") trueVal = targetVessel.capacity -
              targetVessel.targetVolume;
            else if (p.target_property === "capacity") trueVal = targetVessel.targetVolume;
            else if (targetVessel.contents && targetVessel.contents.chemicals &&
              targetVessel.contents.chemicals[p.target_property]) {
              trueVal = targetVessel.contents.chemicals[p.target_property].volume;
            }
          }
        }
        if (abs(userVal - trueVal) > p.tolerance) {
          this.addPenalty(`obs_${p.title}`, p.penalty_points, `Inaccurate observation for
            ${p.title}. Expected approx: ${trueVal.toFixed(2)}`);
        }
        this.studentObservations = this.studentObservations || {};
        this.studentObservations[p.title] = userVal;
      }
    }
  }

  // Trigger Generic Calculation Prompts Sequentially
  if (m.calculation_prompts && m.calculation_prompts.length > 0) {
    for (let c of m.calculation_prompts) {
      let val = window.prompt(`CALCULATION:\n${c.description}`);
      if (val !== null && val.trim() !== "") {
        let userVal = parseFloat(val);
        let parsedFormula = c.formula;
        if (this.studentObservations) {
          for (let key in this.studentObservations) {

```

```

        // simple variable substitution loop (replace var name with float)
        let re = new RegExp(`\\b${key}\\b`, 'g');
        parsedFormula = parsedFormula.replace(re, this.studentObservations[key]);
    }
}
let trueVal = NaN;
try { trueVal = eval(parsedFormula); } catch(e) { console.error("Formula eval
failed due to missing/invalid variables:", e); }

if (isNaN(trueVal)) {
    console.error("Formula misconfigured by instructor. Skipping penalty for " +
c.title);
} else if (abs(userVal - trueVal) > c.tolerance) {
    this.addPenalty(`calc_${c.title}`, c.points, `Incorrect calculation for ${c.title}.
The correct derived value was roughly ${trueVal.toFixed(3)}.`);
}
this.studentCalculations = this.studentCalculations || {};
this.studentCalculations[c.title] = userVal;
}
}
}

this.completedIds.add(id);
sessionMarks += m.points;
currentStepIndex = Math.min(this.milestones.length - 1, this.completedIds.size);
console.log("Milestone Achieved:", m.desc);

// Check if this was the last milestone in the sequence
if (this.completedIds.size >= this.milestones.length && this.milestones.length > 0) {
    alert("All milestones complete! Committing final experiment results.");
    this.saveResults(); // Autonomous generic dispatch
}
}

saveResults() {
    const payload = {
        name: experimentData.name,
        totalScore: sessionMarks,
        observations: this.studentObservations || {},
        calculations: this.studentCalculations || {},
        log: penalties.join(" | ")
    };
};

fetch("/save_lab_report/", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(payload)
})
.then(res => res.json())
.then(data => {

```

```

        alert(`Assessment Complete!\nFinal Score: ${sessionMarks}/100\nYour report has
        been submitted to the instructor.`);
    })
    .catch(err => console.error("Save error:", err));
}
}

```

```

// --- FIX 2: Initialize only ONE manager ---
const manager = new MarkingManager(experimentData);
function idIsDone(id) { return manager.completedIds.has(id); }

```

```

function getLabSurfaces() {
    const scaleX = width / 1200;
    const scaleY = height / 700;
    return {
        shelfTop: {
            y: 85 * scaleY,
            minX: 280 * scaleX,
            maxX: 950 * scaleX,
            shadowAlpha: 30
        },
        shelfBottom: {
            y: 175 * scaleY,
            minX: 280 * scaleX,
            maxX: 950 * scaleX,
            shadowAlpha: 30
        },
        table: {
            y: 335 * scaleY,
            minX: 50 * scaleX,    // Wider coverage
            maxX: width * 0.99,
            shadowAlpha: 45
        }
    };
}

```

```

function updateLabSurfaces() {
    labSurfaces = getLabSurfaces();
}

```

```

// Big vs Small apparatus classification
const BIG_APPARATUS = [
    'burette', 'bunsen_burner', 'wash_bottle', 'hotplate', 'balance',
    'liebig_condensor', 'separatory_funnel', 'pH_meter', 'meltingpoint_apparatus',
    'common_stand'
];

```

```

const SMALL_APPARATUS = [
  'beaker', 'pipette', 'bottle', 'funnel', 'conical_flask',
  'volumetric_flask', 'crucible', 'TLC_plate', 'burette_tube'
];

function getResponsivePositions() {
  const scaleX = width / 1200;
  const scaleY = height / 700;
  const scale = min(scaleX, scaleY);

  return {
    burette_stand: { x: 540 * scale, y: 260 * scale },
    sizes: {
      beaker: { w: 120 * scale * sizeMultiplier, h: 140 * scale * sizeMultiplier },
      pipette: { w: 150 * scale * sizeMultiplier, h: 130 * scale * sizeMultiplier },
      burette: { w: 200 * scale * sizeMultiplier, h: 360 * scale * sizeMultiplier },
      bottle: { w: 120 * scale * sizeMultiplier, h: 180 * scale * sizeMultiplier },
      balance: { w: 200 * scale * sizeMultiplier, h: 250 * scale * sizeMultiplier },
      crucible: { w: 100 * scale * sizeMultiplier, h: 100 * scale * sizeMultiplier },
      hotplate: { w: 200 * scale * sizeMultiplier, h: 150 * scale * sizeMultiplier },
      liebig_condensor: { w: 150 * scale * sizeMultiplier, h: 160 * scale * sizeMultiplier },
      meltingpoint_apparatus: { w: 200 * scale * sizeMultiplier, h: 250 * scale *
sizeMultiplier },
      pH_meter: { w: 200 * scale * sizeMultiplier, h: 250 * scale * sizeMultiplier },
      separatory_funnel: { w: 200 * scale * sizeMultiplier, h: 360 * scale * sizeMultiplier },
      TLC_plate: { w: 100 * scale * sizeMultiplier, h: 120 * scale * sizeMultiplier },
      conical_flask: { w: 120 * scale * sizeMultiplier, h: 150 * scale * sizeMultiplier },
      volumetric_flask: { w: 120 * scale * sizeMultiplier, h: 160 * scale * sizeMultiplier },
      funnel: { w: 80 * scale * sizeMultiplier, h: 80 * scale * sizeMultiplier },
      wash_bottle: { w: 120 * scale * sizeMultiplier, h: 180 * scale * sizeMultiplier },
      bunsen_burner: { w: 120 * scale * sizeMultiplier, h: 180 * sizeMultiplier * scale },
      burette_tube: { w: 100 * scale * sizeMultiplier, h: 360 * scale * sizeMultiplier },
      common_stand: { w: 180 * scale * sizeMultiplier, h: 560 * scale * sizeMultiplier }
    }
  };
}

function preload() {
  imgLabBg = loadImage('/static/images/new-lab-bg-1.png');
  imgBeaker = loadImage('/static/img/catalog/beaker.png');
  imgBottle = loadImage('/static/img/catalog/bottle.png');
  imgBurette = loadImage('/static/img/catalog/burette.png');
  imgPipette = loadImage('/static/img/catalog/pipette.png');
  imgConical = loadImage('/static/img/catalog/conical_flask.png');
  imgVolumetric = loadImage('/static/img/catalog/volumetric_flask.png');
  imgFunnel = loadImage('/static/img/catalog/funnel.png');
  imgWash = loadImage('/static/img/catalog/wash_bottle.png');
  imgBunsen = loadImage('/static/img/catalog/bunsen_burner.png');
  imgBalance = loadImage('/static/img/catalog/balance.png');
  imgCrucible = loadImage('/static/img/catalog/crucible.png');

```

```

imgHotplate = loadImage('/static/img/catalog/hotplate.png');
imgLiebig = loadImage('/static/img/catalog/liebig_condensor.png');
imgMeltingPoint = loadImage('/static/img/catalog/meltingpoint_apparatus.png');
imgPHMeter = loadImage('/static/img/catalog/pH_meter.png');
imgSepFunnel = loadImage('/static/img/catalog/separatory_funnel.png');
imgTLC = loadImage('/static/img/catalog/TLC_plate.png');
imgStand = loadImage('/static/img/catalog/stand.png');
imgBuretteTube = loadImage('/static/img/catalog/burette-single.png');
}

function setup() {
  const root = document.getElementById('simulation-canvas');
  const w = root ? root.clientWidth : window.innerWidth;
  const h = root ? root.clientHeight : window.innerHeight - 48;

  let canvas = createCanvas(w, h);
  canvas.parent('simulation-canvas');

  vessels = {};
  studentVolume = 0;
  phStage = 0;
  beakerTargetVol = 0;
  pipetteTargetVol = 0;
  buretteTargetVol = 0;

  updateLabSurfaces();

  updateResponsivePositions();

  // Dynamic Apparatus Loading
  let dynamicApparatus = [];
  if (experimentData && experimentData.catalogs &&
  experimentData.catalogs.apparatus) {
    dynamicApparatus = experimentData.catalogs.apparatus;
  }
  apparatusCatalog = new LabCatalog({ scale: 0.75, apparatus: dynamicApparatus });
  apparatusCatalog.initSprites({
    beaker: imgBeaker, pipette: imgPipette, bottle: imgBottle,
    burette: imgBurette, conical_flask: imgConical,
    volumetric_flask: imgVolumetric, funnel: imgFunnel,
    wash_bottle: imgWash, bunsen_burner: imgBunsen,
    balance: imgBalance, crucible: imgCrucible,
    hotplate: imgHotplate, liebig_condensor: imgLiebig,
    meltingpoint_apparatus: imgMeltingPoint, pH_meter: imgPHMeter,
    separatory_funnel: imgSepFunnel, TLC_plate: imgTLC,
    burette_tube: imgBuretteTube, common_stand: imgStand
  });

  // Dynamic Chemical Loading
  let dynamicChemicals = [];

```

```

if (experimentData && experimentData.catalogs &&
experimentData.catalogs.chemicals) {
  dynamicChemicals = experimentData.catalogs.chemicals.map(c => {
    // Parse hex color "#RRGGBB" to [R, G, B]
    let r = 200, g = 200, b = 200;
    if (c.color && c.color.startsWith('#')) {
      let hex = c.color.replace('#', '');
      if (hex.length >= 6) {
        r = parseInt(hex.substring(0, 2), 16);
        g = parseInt(hex.substring(2, 4), 16);
        b = parseInt(hex.substring(4, 6), 16);
      }
    }
    return {
      id: c.name, // Use name as ID for matching rules
      label: c.name,
      name: c.name,
      formula: c.formula,
      conc: c.molarity ? c.molarity + 'M' : '',
      color: [r, g, b],
      is_indicator: c.is_indicator,
      low_ph_color: c.low_ph_color,
      high_ph_color: c.high_ph_color,
      transition_ph_range: c.transition_ph_range
    };
  });
} else {
  // Fallback if no DB catalogs present
  dynamicChemicals = [
    { id: 'Sodium Carbonate + Bicarbonate', label: '25% Na2CO3+NaHCO3', name:
'Sodium Carbonate + Bicarbonate', formula: 'Na2CO3 + NaHCO3', conc: '25%', color:
[220, 180, 100], is_indicator: false },
    { id: 'Hydrochloric Acid', label: '0.1M HCl (Burette)', name: 'Hydrochloric Acid',
formula: 'HCl', conc: '0.1M', color: [255, 120, 80], is_indicator: false },
    { id: 'Phenolphthalein', label: 'Phenolphthalein', name: 'Phenolphthalein', formula:
'C20H14O4', conc: '', color: [255, 180, 220], is_indicator: true },
    { id: 'Methyl Orange', label: 'Methyl Orange', name: 'Methyl Orange', formula:
'C14H14N3NaO3S', conc: '', color: [255, 160, 60], is_indicator: true },
    { id: 'Distilled Water', label: 'Distilled Water', name: 'Distilled Water', formula: 'H2O',
conc: '', color: [200, 220, 255] }
  ];
}

chemicalCatalog = new ChemicalCatalog(dynamicChemicals);

// Expose global physics reactions from dynamic payload
window.CHEMICAL_REACTIONS = (experimentData && experimentData.catalogs) ?
experimentData.catalogs.reactions : [];

// AUTO-SPAWN from Scene Builder initial_state

```

```

    spawnFromInitialState();
  }

function spawnFromInitialState() {
  if (!experimentData || !experimentData.initial_state ||
!experimentData.initial_state.length) {
    console.log("No initial_state defined for this experiment.");
    return;
  }

  console.log("Spawning initial state:", experimentData.initial_state);

  experimentData.initial_state.forEach((item, index) => {
    const type = item.type;
    if (!type) return;

    // 1. Determine spawn location based on Scene Builder selection
    let spawnSurface = labSurfaces.table; // default
    if (item.location === 'shelfTop') spawnSurface = labSurfaces.shelfTop;
    else if (item.location === 'shelfBottom') spawnSurface = labSurfaces.shelfBottom;

    // 2. Calculate position
    // NEW: Prioritize item.x if provided by the builder, otherwise auto-arrange
    let targetX;
    if (item.x !== undefined && item.x !== null) {
      targetX = item.x;
    } else {
      const surfaceWidth = spawnSurface.maxX - spawnSurface.minX;
      const spacing = surfaceWidth / (experimentData.initial_state.length + 1);
      targetX = spawnSurface.minX + spacing * (index + 1);
    }

    const size = currentPositions.sizes[type] || currentPositions.sizes.beaker;
    const targetY = spawnSurface.y - (size.h / 2) - 5;
    const pos = findCollisionFreePosition(targetX, targetY, type);

    // 3. Create the vessel
    const id = nextId(type);
    let v = makeVessel(id, pos.x, pos.y, size.w, size.h, type, 'Empty', type, 0, 100);

    // 4. Set type-specific defaults
    if (type === 'beaker') {
      v.capacity = 250;
      v.title = '250 mL Beaker';
    } else if (type === 'conical_flask') {
      v.capacity = 250;
      v.title = 'Conical Flask';
    } else if (type === 'pipette') {
      v.capacity = 25;
      v.title = '25 mL Pipette';
    }
  });
}

```

```

} else if (type === 'burette_tube') {
  v.capacity = 50;
  v.title = '50 mL Burette Tube';
  v.isBurette = true;
  v.mountedTo = null;
  v.clampOffset = 0;
} else if (type === 'common_stand') {
  v.title = 'Common Stand';
} else if (type === 'funnel') {
  v.title = 'Filter Funnel';
} else if (type === 'wash_bottle') {
  v.chem = 'Distilled Water';
  v.capacity = 250;
  v.title = 'Wash Bottle';
} else if (type === 'bottle') {
  v.capacity = 250;
  v.title = 'Empty Bottle';
} else if (type === 'volumetric_flask') {
  v.capacity = 250;
  v.title = '250 mL Volumetric Flask';
} else if (type === 'burette') {
  v.capacity = 50;
  v.title = '50 mL Burette';
  v.targetVolume = 0;
  v.contents.hcl_vol = 0;
} else if (type === 'bunsen_burner') {
  v.title = 'Bunsen Burner';
} else if (type === 'balance') {
  v.title = 'Analytical Balance';
  v.tareOffset = 0;
  v.rawWeight = 0;
  v.displayWeight = 0;
  v.mass = 0;
} else if (type === 'crucible') {
  v.capacity = 20;
  v.title = 'Porcelain Crucible';
} else if (type === 'hotplate') {
  v.title = 'Digital Hotplate';
  v.temperature = 25;
  v.heating = false;
} else if (type === 'pH_meter') {
  v.title = 'Digital pH Meter';
  v.reading = 7.0;
} else if (type === 'dropper') {
  v.capacity = 5;
  v.title = 'Glass Dropper';
} else {
  v.title = type.replace(/_/g, ' ');
}

```

```

// 5. Fill with chemical - Support both new nested (initialContents) and old flat
(chem/vol) structure
let chemName = null;
let chemVol = 0;

if (item.initialContents && item.initialContents.type) {
  chemName = item.initialContents.type;
  chemVol = item.initialContents.volume || 0;
} else if (item.chem) {
  // Fallback for flat structure
  chemName = item.chem;
  chemVol = item.vol || 0;
}

if (chemName) {
  // Look up color from the chemical catalog
  let chemColor = [200, 220, 255];
  if (chemicalCatalog && chemicalCatalog.chemicals) {
    const found = chemicalCatalog.chemicals.find(c => c.name === chemName || c.id
=== chemName);
    if (found && found.color) {
      chemColor = found.color;
    }
  }

  v.chem = chemName;
  v.chemicalId = chemName;
  v.color = chemColor;
  v.volume = chemVol;
  v.targetVolume = chemVol;

  // For bottles, mark as chemical bottle
  if (type === 'bottle') {
    v.isChemical = true;
    v.title = chemName;
  }

  // Initialize dynamic chemical dictionary
  v.contents.chemicals[chemName] = { volume: chemVol, color: chemColor };
}

vessels[v.id] = v;
console.log(` Auto-spawned: ${v.title} at (${Math.round(pos.x)},
${Math.round(pos.y)}) on ${item.location || 'table'}`);
});

// NEW: Auto-Mount Sweep - Link tubes to stands if they are near each other on spawn
const stands = Object.values(vessels).filter(v => v.type === 'common_stand');
const tubes = Object.values(vessels).filter(v => v.type === 'burette_tube');

```

```

tubes.forEach(tube => {
  // Find a stand that is horizontally aligned (within 40px)
  const matchingStand = stands.find(s => Math.abs(s.x - tube.x) < 40);
  if (matchingStand) {
    tube.mountedTo = matchingStand.id;
    // Snap tube to stand position
    tube.x = matchingStand.x;
    tube.y = matchingStand.y - (matchingStand.h * 0.1);
    console.log(`Auto-mounted ${tube.id} to stand ${matchingStand.id}`);
  }
});
}

function windowResized() {
  const root = document.getElementById('simulation-canvas');
  const w = root ? root.clientWidth : window.innerWidth;
  const h = root ? root.clientHeight : window.innerHeight - 48;
  resizeCanvas(w, h);

  updateLabSurfaces(); // Recalculate surfaces

  if (currentPositions) {
    Object.values(vessels).forEach(v => {
      const size = currentPositions.sizes[v.type] || currentPositions.sizes.beaker;
      v.w = size.w;
      v.h = size.h;
    });
  }
}

function updateResponsivePositions() {
  currentPositions = getResponsivePositions();
}

function askCapacity(type) {
  const options = {
    beaker: [50, 100, 250, 500],
    pipette: [10, 25],
    burette: [25, 50],
    volumetric_flask: [100, 250, 500],
    separatory_funnel: [250, 500]
  }[type] || [];

  if (!options.length) return null;

  const input = window.prompt(`Choose ${type} capacity (mL): ` + options.join(', '));
  if (input === null) return null;
  const value = parseFloat(input);
  return options.includes(value) ? value : null;
}

```

```

function smartSpawnPosition(type) {
  // Use dynamic lab surfaces (no more fixed LAB_ZONES!)
  if (!labSurfaces) return { x: width / 2, y: height / 2 };

  // ALL items spawn on specific surfaces now
  const isBig = BIG_APPARATUS.includes(type);

  let spawnSurface;
  if (isBig) {
    spawnSurface = labSurfaces.table;
  } else {
    spawnSurface = Math.random() > 0.5 ? labSurfaces.shelfTop :
labSurfaces.shelfBottom;
  }

  // Spawn range
  const minX = spawnSurface.minX + 50;
  const range = (spawnSurface.maxX - spawnSurface.minX) * 0.8;
  const targetX = minX + random(0, range);

  // PREDICT HEIGHT to spawn ABOVE the surface (so logic/physics snaps it down)
  let size = currentPositions?.sizes[type] || { h: 100, w: 100 };
  let verticalSize = (type === 'burette_tube') ? size.w : size.h; // Tube lays flat on spawn
  const targetY = spawnSurface.y - (verticalSize / 2) - 5; // Spawn 5px above snap point

  return findCollisionFreePosition(targetX, targetY, type);
}

function findCollisionFreePosition(targetX, targetY, type) {
  let attempts = 0;
  const scaleX = width / 1200;
  const scaleY = height / 700;

  while (attempts < 20) {
    let testX = targetX + random(-100, 100);
    let testY = targetY; // Keep Y fixed to surface level

    // Keep within lab surface bounds
    /*
    const isBig = BIG_APPARATUS.includes(type);
    const bounds = isBig ? labSurfaces.table : labSurfaces.shelf;

    if (testX < bounds.minX || testX > bounds.maxX || testY > bounds.y + 20) {
      attempts++;
      continue;
    }
    */
  }
}

```

```

    // Check collision with other vessels
    let collision = false;
    Object.values(vessels).forEach(other => {
        if (dist(testX, testY, other.x, other.y) < 60) {
            collision = true;
        }
    });

    if (!collision) return { x: testX, y: testY };
    attempts++;
}

// Fallback: surface center
return {
    x: constrain(targetX, labSurfaces.table.minX + 50, labSurfaces.table.maxX - 50),
    y: labSurfaces.table.y
};
}

function applySloshPhysics(v) {
    // Calculate horizontal velocity for sloshing
    let horizontalMove = v.x - (v.lastX || v.x);
    v.lastX = v.x;

    // Target tilt is based on speed (inertia)
    let targetTilt = constrain(horizontalMove * 0.12, -0.6, 0.6);

    // Spring-damping physics for the liquid surface
    let springForce = (targetTilt - v.tilt) * 0.15;
    v.tiltVel += springForce;
    v.tiltVel *= 0.82; // Damping (makes it stop sloshing eventually)
    v.tilt += v.tiltVel;

    // Return to level if not moving
    if (!v.dragging) {
        v.tilt = lerp(v.tilt, 0, 0.05);
    }

    // Turbulence Decay (Ripples settle down)
    if (v.turbulence > 0) {
        v.turbulence = lerp(v.turbulence, 0, 0.05);
        if (v.turbulence < 0.01) v.turbulence = 0;
    }
}

function makeVessel(id, x, y, w, h, title, chem, vtype, vol, cap) {
    return {
        // --- BASIC IDENTITY ---
        id: id,

```

```

    type: vtype,
    title: title,
    chem: chem,
    chemicalId: null,
    x: x, y: y, w: w, h: h,
    vx: 0, vy: 0,
    lastX: x, tilt: 0, tiltVel: 0,
    turbulence: 0, // NEW: For liquid surface ripples
    surface: null,

    volume: vol,
    targetVolume: vol,
    capacity: cap,

    hasFunnel: false,
    tiltAngle: 0,
    isDraining: false,
    dragging: false,
    isOnBalance: false,
    vanish: 1.0,
    color: [200, 220, 255, 150],

    contents: {
      chemicals: {},
      indicators: {},
      indicatorsAdded: [],
      isRinsed: false,
      isContaminated: false,
      titrant_vol: 0, // Tracking cumulative titrant added
      mixture_vol: 0, // Tracking initial analyte volume
      pH: 7.0,
      temperature: 25.0,
      solidMass: 0
    }
  };
}

function makeResponsiveVessel(id, type) {
  if (!currentPositions) return null;
  const spawnPos = smartSpawnPosition(type);
  const size = currentPositions.sizes[type] || currentPositions.sizes.beaker;
  return makeVessel(id, spawnPos.x, spawnPos.y, size.w, size.h, type, 'Empty', type, 0,
100);
}

function getTitrationColor(v) {
  const c = v.contents;
  if (!c) return v.color || [200, 220, 255, 100];

  // ABORT: If no mixture detected via tracking or chemical search

```

```

let hasAnalyte = (c.mixture_vol > 0.1);
if (!hasAnalyte && c.chemicals) {
  // Fallback: check chemical dictionary for common analyte terms
  hasAnalyte = Object.keys(c.chemicals).some(name =>
    name.toLowerCase().includes('mixture') || name.toLowerCase().includes('analyte') ||
name.toLowerCase().includes('carbonate')
  );
}

if (!hasAnalyte) return v.color || [200, 220, 255, 100];

// Default values if not defined in UI
let v1_ph = 8.3;
let v2_ph = 4.0;
let v1_color = experimentData?.v1_color || "#FFC0CB"; // Default Pink
let v2_color = experimentData?.v2_color || "#FF4500"; // Default Orange

// Extract pH thresholds from milestone rules
if (experimentData && experimentData.milestones) {
  const m1 = experimentData.milestones.find(m => m.id === "reach_v1" || m.id ===
"endpoint_1");
  if (m1 && m1.rules) {
    const r = m1.rules.find(rule => rule.target_property === "pH");
    if (r) v1_ph = r.value;
  }
  const m2 = experimentData.milestones.find(m => m.id === "reach_v2" || m.id ===
"endpoint_2");
  if (m2 && m2.rules) {
    const r = m2.rules.find(rule => rule.target_property === "pH");
    if (r) v2_ph = r.value;
  }
}

const isDouble = experimentData?.type === 'double_indicator' || (c.indicatorsAdded &&
c.indicatorsAdded.length > 1);

// Base (12) -> V1 (Target) -> V2 (Target)
let currentPH = 12.0;
const targetV1 = experimentData?.targets?.v1 || 10.0;
const targetV2 = experimentData?.targets?.v2 || 25.0;

if (c.titrant_vol < targetV1) {
  currentPH = map(c.titrant_vol, 0, targetV1, 12.0, v1_ph);
} else if (!isDouble) {
  currentPH = map(c.titrant_vol, targetV1, targetV1 + 2.0, v1_ph - 1.0, 2.0, true);
} else if (c.titrant_vol < targetV2) {
  currentPH = map(c.titrant_vol, targetV1, targetV2, v1_ph - 0.3, v2_ph);
} else {
  currentPH = map(c.titrant_vol, targetV2, targetV2 + 2.0, v2_ph - 0.2, 1.5, true);
}

```

```

let baseCol = [230, 230, 250, 100]; // Base liquid
let r=baseCol[0], g=baseCol[1], b=baseCol[2], a=baseCol[3];

if (c.indicatorsAdded && c.indicatorsAdded.length > 0) {
  c.indicatorsAdded.forEach(indName => {
    const chem = chemicalCatalog.chemicals.find(ch => ch.name === indName || ch.id
=== indName);
    if (!chem) return;

    const lowTheme = hexToRgba(chem.low_ph_color || "#FFFFFF00");
    const highTheme = hexToRgba(chem.high_ph_color || "#FFFFFF00");

    // Determine which target color to use based on which indicator was added
    // Logic: If it's the first indicator, approach v1_color. If second, approach v2_color.
    let targetRGB = hexToRgba(v1_color);
    let thresholdPH = v1_ph;

    if (isDouble && c.indicatorsAdded.indexOf(indName) > 0) {
      targetRGB = hexToRgba(v2_color);
      thresholdPH = v2_ph;
    }

    // Calculate factor (1.0 = Basic, 0.0 = Acidic/Endpoint)
    let factor = map(currentPH, thresholdPH - 0.5, thresholdPH + 1.5, 0, 1, true);

    // Blend indicator high color (basic) with target endpoint color
    r = lerp(targetRGB[0], highTheme[0], factor);
    g = lerp(targetRGB[1], highTheme[1], factor);
    b = lerp(targetRGB[2], highTheme[2], factor);
    a = lerp(targetRGB[3], highTheme[3], factor);
  });
}

return [r, g, b, a];
}

```

7 RESULTS

In this chapter, the functional outcomes of the Virtual Chemistry Laboratory Management System (VCLMS) through a hierarchical progression of its core interfaces are presented. The results demonstrate the seamless integration of institutional management with high-fidelity laboratory simulations, structured according to the role-based flow of the application.

7.1 Authentication and Portal Entry

The entry point of the VCLMS provides a secure and intuitive authentication gateway for Students, Staff, and HODs. The portal utilizing Django's robust security framework ensures that users are redirected to their specialized dashboards upon verification of their credentials. The Signup interface is tailored to capture institutional metadata, allowing for the precise categorization of students into their respective streams and academic years, while the Login page maintains a minimalist design to facilitate efficient session initialization.

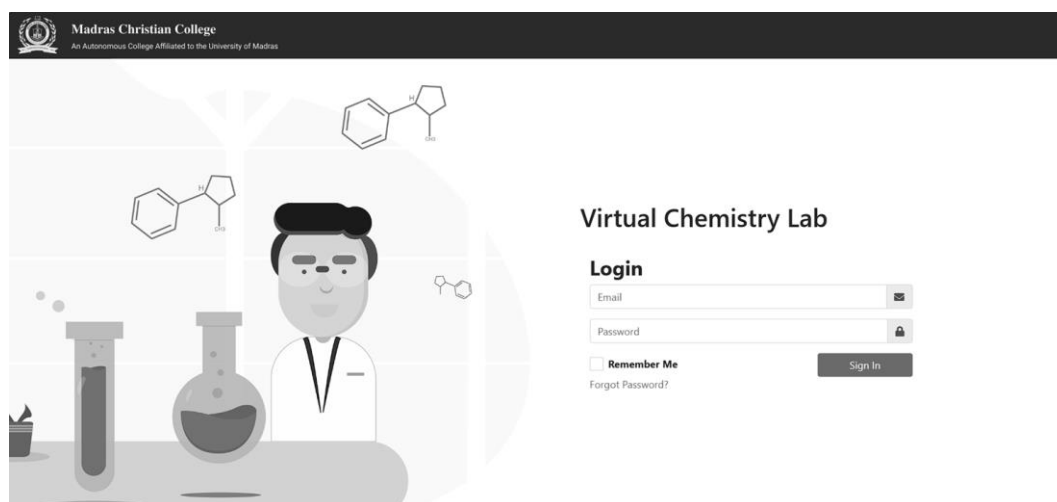


Fig. 7.1. Portal Authentication and Registration Interfaces

7.2 Administrative Oversight

The HOD Dashboard represents the administrative pinnacle of the system, providing a high-level summary of the department's academic health. This interface aggregates total counts of students, faculty, and subjects, presenting them through interactive cards and data visualizations. It serves as the primary command center for academic session management and department-wide performance auditing, allowing the HOD to monitor trends in laboratory proficiency and engagement across the entire institutional registry.

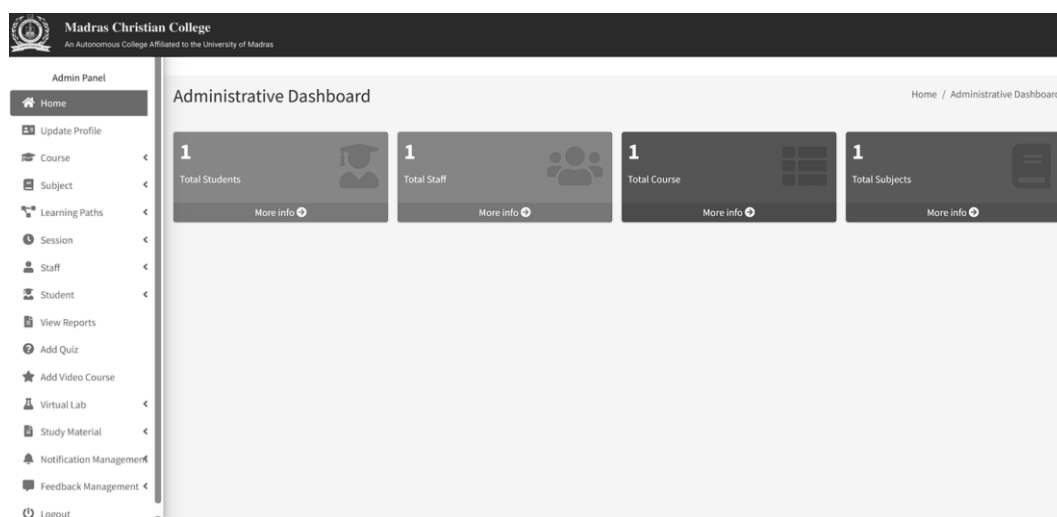
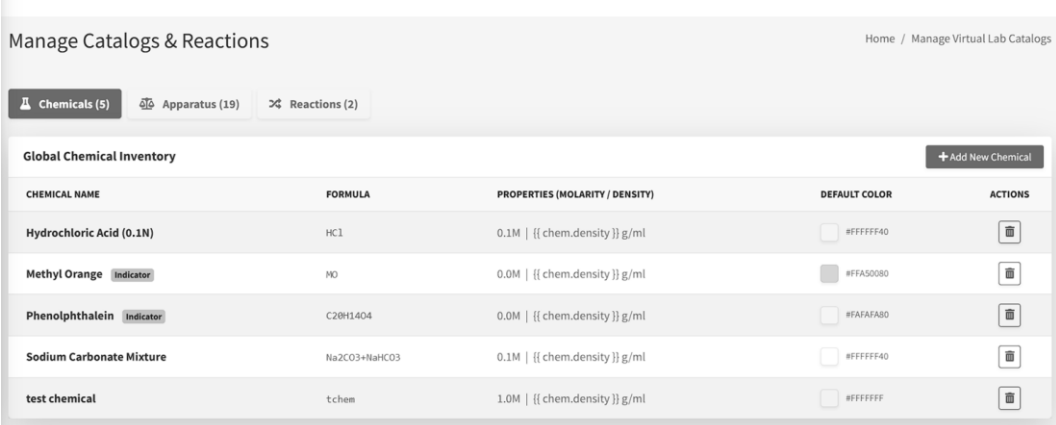


Fig. 7.2. HOD (Admin) Central Management Dashboard

7.3 Chemical Inventory

The Global Chemical Inventory functions as the absolute master repository for all reagents, analytes, and indicators utilized across the institutional environment. It securely catalogs standard chemical nomenclature alongside specific molecular formulas to prevent input redundancies during experiment setups. Furthermore, it permanently registers precise physical properties, such as baseline molarity (e.g., 0.1M) and fluid densities, which the backend utilizes to strictly evaluate the student's mathematical derivations. Crucially, this interface directly ties chemicals to specific rendering metadata, defining default visual states utilizing standard screen Hex codes

(e.g., `#FFA500B0` for Methyl Orange). This ensures seamless, physically accurate color morphing across all simulated experiments regardless of which faculty member authors them.



Manage Catalogs & Reactions Home / Manage Virtual Lab Catalogs

Chemicals (5) Apparatus (19) Reactions (2)

Global Chemical Inventory + Add New Chemical

CHEMICAL NAME	FORMULA	PROPERTIES (MOLARITY / DENSITY)	DEFAULT COLOR	ACTIONS
Hydrochloric Acid (0.1N)	HC1	0.1M {{ chem.density }} g/ml	☐ #FFFFFF40	
Methyl Orange Indicator	HO	0.0M {{ chem.density }} g/ml	☒ #FFA500B0	
Phenolphthalein Indicator	C20H14O4	0.0M {{ chem.density }} g/ml	☐ #FAFAFA80	
Sodium Carbonate Mixture	Na2CO3+NaHCO3	0.1M {{ chem.density }} g/ml	☐ #FFFFFF40	
test chemical	t:chem	1.0M {{ chem.density }} g/ml	☐ #FFFFFF	

Fig. 7.3. Chemical Inventory managed by Admin

7.4 The Digital Apparatus Registry

The Digital Apparatus Registry meticulously manages the structural and geometric inventory of the simulated laboratory, providing standard physical items such as 50mL Burettes and 250mL Conical Flasks. By cataloging precise, predefined coordinate boundaries and collision layers, this module acts as the physical blueprint directly parsed by the `p5.js` rendering engine. It actively secures volumetric capacity limits, ensuring that the engine mathematically recognizes exact overflow points to enforce strict procedural realism. Locking these absolute dimensions at the global administrative level guarantees that students consistently encounter highly standardized UI interactions. This uniformity effortlessly mimics the stringent standardization protocols of a professional, physical chemistry stockroom.

Manage Catalogs & Reactions

Home / Manage Virtual Lab Catalogs

Chemicals (5)

Apparatus (19)

Reactions (2)

Apparatus Engine Catalog

+ Add New Apparatus

APPARATUS NAME	ENGINE TYPE ID	MAX CAPACITY	PHYSICS BEHAVIORS	ACTIONS
Analytical Balance	balance	0.0 ml		
Beaker	beaker	250.0 ml	Volumetric Heatable Pourable	
Bunsen Burner	bunsen_burner	0.0 ml	Heatable	
Burette Tube	burette_tube	50.0 ml	Volumetric Pourable	
Classic Burette	burette	50.0 ml	Volumetric Pourable	
Common Stand	common_stand	0.0 ml		
Condenser	liebig_condensor	0.0 ml	Heatable	
Conical Flask	conical_flask	250.0 ml	Volumetric Heatable Pourable	

Fig. 7.4. Apparatus Engine Management

7.5 Chemical Reaction and Endpoint Logic

The Catalog Management module lets the admin define the exact physics and reactionary behavior. The Admin sets the precise baseline volumes for the simulation and determines the theoretical endpoint indicator targets (such as calculating the precise required shift from Phenolphthalein pink to Methyl Orange red). The GUI absorbs these mathematical limitations and automatically writes the underlying stoichiometric calculation logic. When finalized and published, the system aggregates the Metadata, Apparatus, Chemical, and Reaction data into one unified, encrypted JSON schema that is dynamically processed by the simulation engine at runtime.

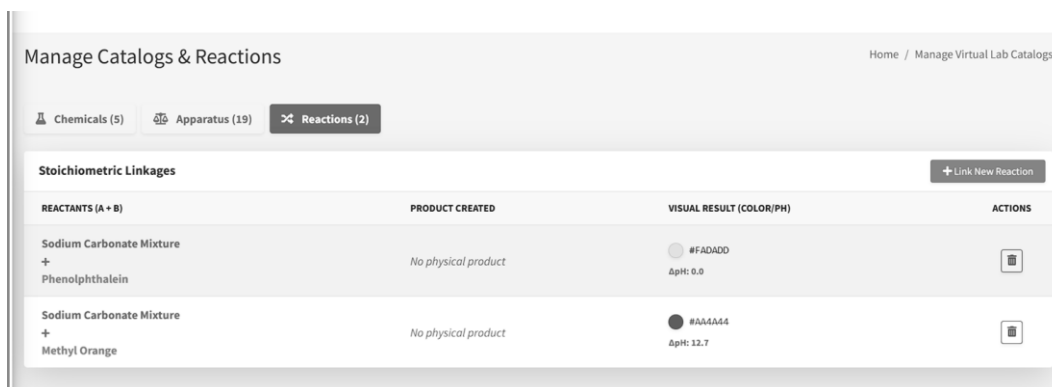


Fig. 7.5. Staff Instructional Dashboard

7.6 Experiment Creation

The foundational step of the No-Code builder involves the Admin defining the global metadata for the new simulation. Through the UI, the Admin assigns the Experiment Title, categorizes it into a specific syllabus stream, and establishes the formal instructions. This data is securely processed via Django forms to create the parent JSON directory structure in the PostgreSQL database, guaranteeing that the experiment is uniquely identifiable across the institutional environment.

Madras Christian College
An Autonomous College Affiliated to the University of Madras

Admin Panel

- Home
- Update Profile
- Course
- Subject
- Learning Paths
- Session
- Staff
- Student
- View Reports
- Add Quiz
- Add Video Course
- Virtual Lab
- Study Material
- Notification Management
- Feedback Management

Add Virtual Lab Experiment

Basic Information

Experiment Title

URL Slug (e.g., simple-titration)

Objective

Principle

Experiment Type
Double Indicator Titration

Simulation supports Double Indicator and Simple Titration templates (see lab engine).

Fig. 7.6. Experiment Initialization by Admin

7.7 Digital Apparatus Configuration

Instead of requiring manual geometric programming to position laboratory equipment on the screen, the No-Code Builder authorizes an intuitive Apparatus Configuration interface. The Admin utilizes dynamic dropdown menus to select the required geometry—such as specifying a 50mL continuous flow Burette, a 250mL Conical Flask, and the necessary volumetric pipettes. The interface automatically calculates initial coordinate properties and collision layers for the chosen items, serializing this directly into the payload so the `p5.js` engine knows exactly how to render the spatial canvas physically.

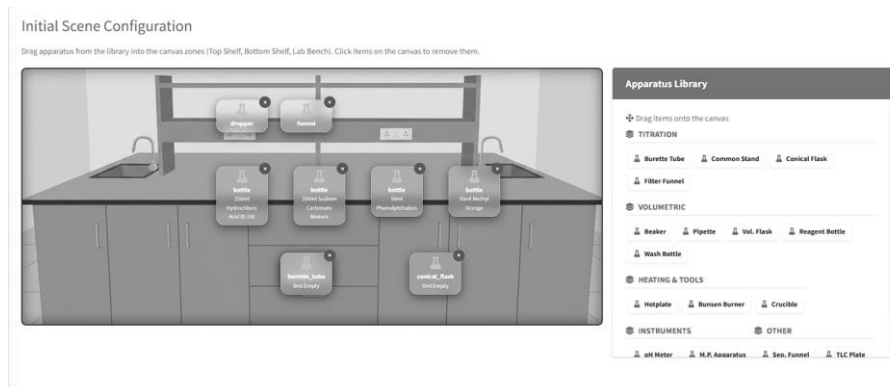


Fig. 7.7. Experiment Initialization by Admin

7.8 Chemical Selection and Assignment

A critical innovation of the authoring studio is the integrated Chemical Selection Modal. Rather than allowing Admins to type free-text—which historically introduces massive database matching errors—the Admin selects reagents directly from secure, pre-cataloged database strings. The Admin officially assigns the 'Titrant' (e.g., 0.1M HCl) directly to the previously authored burette, and the 'Analyte' (e.g., Na₂CO₃) to the conical flask. This 'Material Sync' architecture perfectly binds the physical rendering space to the precise molecular nomenclature required for strict academic validation.

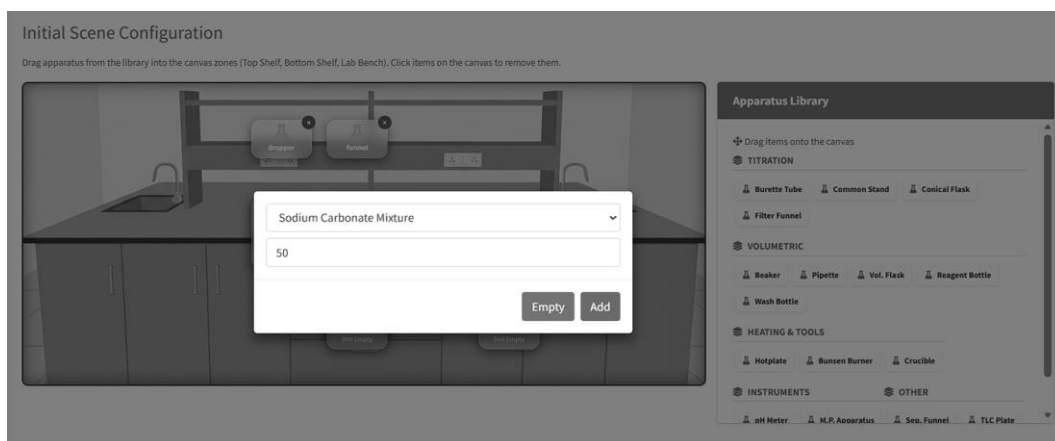


Fig. 7.8. Database-Linked Chemical Assignment Modal

7.9 Instructional Management

The Staff Dashboard is optimized for faculty members to manage their assigned subjects and instructional materials effectively. This interface provides dedicated buttons for uploading study materials, managing video courses, and capturing real-time student attendance. It also serves as the review portal where faculty can inspect student laboratory attempts and procedural logs, ensuring that the transition from theoretical teaching to practical evaluation is tightly controlled and documented for academic credit.

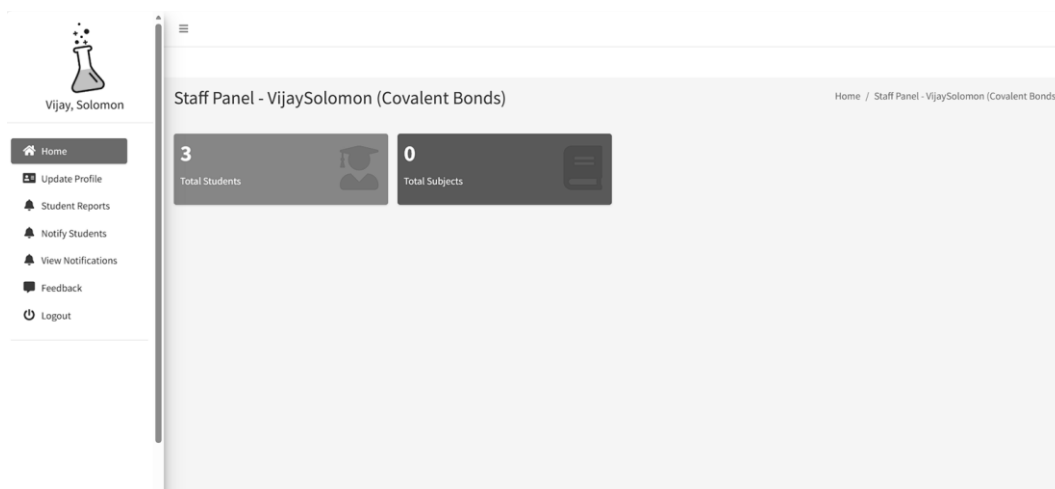


Fig. 7.9. Staff Instructional Dashboard

7.10 Student Learning Dashboard

The Student Dashboard is designed to be the central hub for the student's academic journey, providing access to theoretical resources and historical performance data. The dashboard acts as a preparatory stage, guiding the student to master theoretical concepts before launching the high-stakes virtual laboratory simulation.

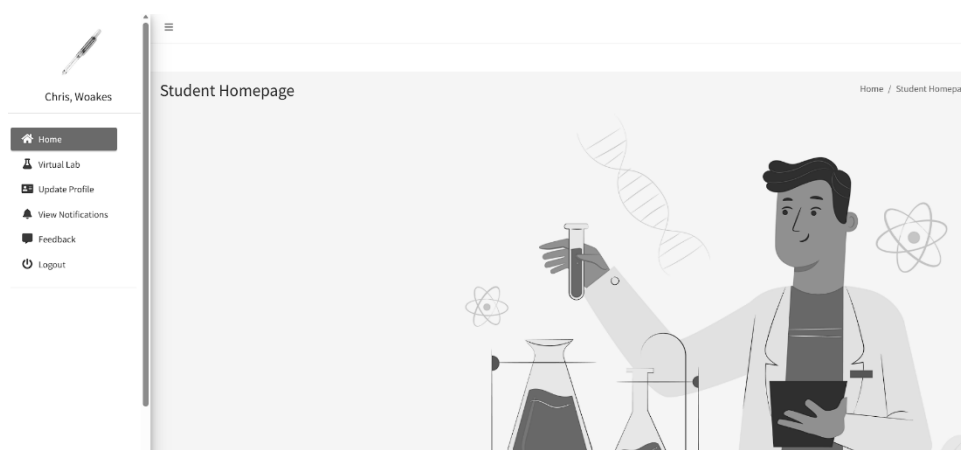


Fig. 7.10. Student Academic Progress Dashboard

7.11 Virtual Chemistry Laboratory

7.11.1 Lab Catalog and Apparatus Setup

The laboratory lifecycle begins with the interactive apparatus catalog, which provides a digital inventory of standardized chemistry equipment. Students must correctly identify and drag-and-drop the required vessels—such as the burette tube and conical flask—onto the specialized laboratory bench. The system performs real-time collision detection and snapping logic, ensuring that the apparatus is positioned according to standardized experimental configurations before any chemical reagents are introduced.



Fig. 7.11.1 Interactive Apparatus Selection and Workspace Setup

7.11.2 Titration Execution

The core titration phase utilizes the p5.js engine to simulate the chemical properties and physical mechanics of volumetric analysis. Students interact with a high-fidelity rendering of the burette, carefully controlling the flow of HCl into the analyte mixture using a dynamic meniscus-level interface. The simulation realistically depicts liquid volumes and the characteristic color transitions of a double-indicator titration (Pink and Yellow), requiring the student to demonstrate the

same level of visual focus and control expected in a physical chemistry department. (Direction for Snap: Capture the active titration in progress with the burette zoomed in, the HCl solution level dropping, and the conical flask showing a vibrant pink color.)

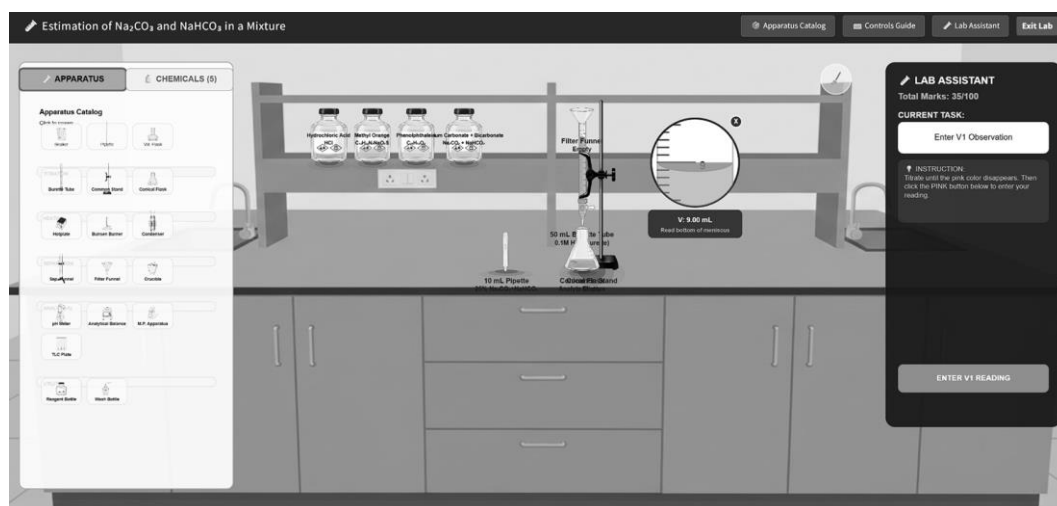


Fig. 7.11.2 Titration Simulation in Progress

7.11.3 Real-time Marking and Penalty Assistant

As the student performs the experiment, the 'Marking Manager' logic engine continuously monitors for procedural accuracy. This sub-section displays a persistent sidebar that alerts the student to their achievements and mistakes in real-time. Procedural errors—such as forgetting to remove the funnel, neglecting to swirl the flask, or failing to zero the burette—are immediately logged as specific penalties. This feature ensures that the student is evaluated based on their professional technique and adherence to laboratory discipline, not just their final quantitative result.



Fig. 7.11.3: Real-time Procedural Marking and Penalty Assistant

7.11.4 Automated Lab Appraisal Report (Final Outcome)

The ultimate outcome of the laboratory session is the generation of an authenticated PDF Appraisal Report. This formal document aggregates all procedural marks, observation accuracy scores, and calculation results into a single professional record. It includes a detailed timestamp, a procedural "Penalty Log" for student review, and the final grade awarded by the marking engine. This report is archived for institutional credit and provides the student with a tangible proof of their practical competence in volumetric analysis. (Direction for Snap: Provide a clear, full-screen screenshot of the finalized PDF report, emphasizing the 'Scoring Summary' and the 'Calculated Experimental Results' table.)

STUDENT REPORT CARD

Name:	Chris Woakes
Email:	chris@gmail.com
Student ID:	4
Course:	Covalent Bonds

Virtual Lab Performance

Experiment	V1	V2	Calc Na_2CO_3	Calc NaHCO_3	Score
Estimation of Na_2CO_3 and NaHCO_3 in a Mixture	10.7 mL	25.2 mL	0.104 g	0.156 g	57/100
Estimation of Na_2CO_3 and NaHCO_3 in a Mixture	10.4 mL	23.7 mL	0.102 g	0.107 g	77/100
Estimation of Na_2CO_3 and NaHCO_3 in a Mixture	10.7 mL	23.8 mL	0.67 g	0.76 g	55/100
Estimation of Na_2CO_3 and NaHCO_3 in a Mixture	11.0 mL	26.2 mL	0.78 g	0.0 g	53/100
Estimation of Na_2CO_3 and NaHCO_3 in a Mixture	9.0 mL	26.6 mL	0.067 g	0.209 g	67/100

Procedural Mistakes & Penalties

Description of Mistake
• -10: Inaccurate V1 observation. Entered: 9, Actual: 10.00
• -10: Titrating without swirling the flask regularly.
• -13: Incorrect Mass calculation for Sodium Bicarbonate.

Academic Quiz Results

Quiz Name	Subject	Marks	Percentage
-----------	---------	-------	------------

Final Grade Calculation

Assessment Type	Score
Virtual Lab Simulation	67
Manual Review Score	10
Quiz Total	0
TOTAL CONSOLIDATED MARKS	77

Fig. 7.11.4 Final Authenticated PDF Lab Appraisal Report

8 CONCLUSION

8.1 Scope of the Project

The primary operational scope of this project was to definitively design, engineer, and deploy a comprehensive Virtual Chemistry Laboratory Management System (VCLMS) that shatters the restrictive boundaries between traditional classroom theory and highly analytical practical execution. The final deployed platform successfully constitutes a risk-free, infinitely scalable, and highly available web-based architecture.

It pioneers an advanced `p5.js` simulation engine executing flawless real-world chemical interactions, fluid draining mechanics, and dynamic stoichiometric indicator visualization. By meticulously engineering the "No-Code Experiment Configuration Builder," the VCLMS has completely decoupled curriculum authoring from heavy software development constraints, empowering chemistry faculty to configure exact chemical matches and mathematical validations instantly via the user interface. Fully integrated with a permanent, robust PostgreSQL backend database, the deployment strictly enforces exact cataloged evaluation logic, ensuring that students are relentlessly assessed on professional laboratory discipline—not just theoretical calculation. Ultimately, VCLMS demonstrates the highest capability of modern educational technology, forging a seamless pipeline from theory, to rigorous manual observation practice, down to authenticated performance certification without geographical or financial limitations.

8.2 Future Enhancements

The advanced architecture of the VCLMS is inherently structured as an infinitely scalable pedagogical foundation that can be expanded immediately into several intensive technical frontiers:

- **Intelligent LLM Generation:** Integrating advanced generative Artificial Intelligence (LLMs) into the No-Code Builder, allowing faculty to simply type a prompt (e.g., "Create a difficult weak acid titration") and having the NLP service automatically generate the dynamic JSON configuration payload and chemical property limits.
- **Virtual Reality (VR) and AR Spatial Hand-Tracking:** Transitioning the highly mathematical 2D 'p5.js' calculations into a fully immersive, 3-Dimensional WebXR or Virtual Reality namespace. This would force students to physically manipulate spatial chemistry apparatus using hand-tracking devices, drastically closing the gap between digital simulation and authentic physical laboratory motor skills.
- **Real-time Synchronous Checkpoints:** Integrating sophisticated WebSockets to facilitate massively multi-user, real-time sessions where a faculty proctor can live-stream themselves entering a student's private virtual lab to provide synchronous, live-action procedural guidance during a high-stakes exam setup.
- **Advanced Thermodynamic Reaction Modeling:** Further upgrading the core mathematical engine to physically simulate absolute real-time thermodynamic properties—such as mapping localized endothermic/exothermic temperature spikes on the canvas that would rapidly affect chemical reaction phase-rates or fluid volumes dynamically.

**VIRTUAL CHEMISTRY LABORATORY AND
LEARNING MANAGEMENT SYSTEM (VCLMS)**

A Project Report

submitted in partial fulfillment of the requirements

for the award of the degree of

MASTER OF COMPUTER APPLICATIONS

of the University of Madras

by

AVINASH N



**Department of Computer Science
Madras Christian College (Autonomous)
Tambaram, Madras 600 059**

April 2026

TABLE OF CONTENTS

1 INTRODUCTION	1
1.1 ORGANIZATION PROFILE.....	1
1.2 PROJECT OVERVIEW	1
1.3 EXISTING SYSTEM.....	3
1.4 PROPOSED SYSTEM	4
1.5 SYSTEM CONFIGURATION	5
1.5.1 Hardware Configuration.....	5
1.5.2 Software Configuration.....	5
1.5 Objectives of the Project	6
2 BACKGROUND STUDY	7
2.1 INTEGRATING VIRTUAL ENVIRONMENTS IN LEARNING MANAGEMENT SYSTEMS.....	7
2.2 PEDAGOGICAL BENEFITS AND COGNITIVE LOAD IN VIRTUAL LABS	7
2.3 THE ROLE OF REAL-TIME INTERACTIVITY AND FEEDBACK	8
2.4 TECHNICAL STANDARDS FOR MODERN WEB-BASED SIMULATIONS	9
2.5 EVOLUTION OF AUTHORING TOOLS IN EDUCATIONAL SIMULATIONS (THE NO- CODE PARADIGM).....	9
3 SYSTEM DESIGN.....	11
3.1 MODULE DESCRIPTIONS.....	11
3.1.1 Authentication & Role-Based Access Control (RBAC) Module	11
3.1.2 No-Code Experiment Configuration Module.....	11
3.1.3 Virtual Laboratory Simulation Module.....	12
3.1.4 Academic Management (LMS) Module.....	12
3.2 WORKFLOW OF THE PROPOSED SYSTEM.....	13
3.2.1 Laboratory Execution Sub-module	13
3.2.2 Calculation & Results Sub-module	14
4 DATABASE DESIGN	16

4.1 USERS TABLE.....	16
4.2 EXPERIMENT CONFIGURATION TABLE	17
4.3 VIRTUAL LAB SUBMISSION TABLE	17
4.4 ATTENDANCE TABLE	18
5 MODULE DESIGN	19
5.1 AUTHENTICATION AND IDENTITY MANAGEMENT MODULE	19
5.2 VIRTUAL LABORATORY MODULE (THE SIMULATION CORE)	19
5.2.1 <i>Real-Time Simulation Engine</i>	19
5.2.2 <i>Apparatus Setup and Handling Logic</i>	20
5.2.3 <i>Titration Execution and Observation</i>	20
5.2.4 <i>Marking Manager and Penalty Logic</i>	20
5.3 ACADEMIC CONTENT AND LEARNING MODULE.....	21
5.4 ASSESSMENT AND MATHEMATICAL VERIFICATION MODULE	21
5.5 NO-CODE EXPERIMENT BUILDER MODULE	22
6 CODING	23
7 RESULTS.....	43
7.1 AUTHENTICATION AND PORTAL ENTRY	43
7.2 ADMINISTRATIVE OVERSIGHT.....	44
7.3 CHEMICAL INVENTORY	44
7.4 THE DIGITAL APPARATUS REGISTRY	45
7.5 CHEMICAL REACTION AND ENDPOINT LOGIC	46
7.6 EXPERIMENT CREATION.....	47
7.7 DIGITAL APPARATUS CONFIGURATION	48
7.8 CHEMICAL SELECTION AND ASSIGNMENT	48
7.9 INSTRUCTIONAL MANAGEMENT.....	49
7.10 STUDENT LEARNING DASHBOARD	50
7.11 VIRTUAL CHEMISTRY LABORATORY	51
7.11.1 <i>Lab Catalog and Apparatus Setup</i>	51
7.11.2 <i>Titration Execution</i>	51

7.11.3 Real-time Marking and Penalty Assistant.....	52
7.11.4 Automated Lab Appraisal Report (Final Outcome).....	53
8 CONCLUSION	55
8.1 SCOPE OF THE PROJECT	55
8.2 FUTURE ENHANCEMENTS.....	56

LIST OF TABLES

table 4.1. User Table.....	16
Table 4.2. Experiment Configuration Table.....	17
Table 4.3. Virtual Lab Submission Table.....	18
Table 4.4. Attendance Table.....	18

LIST OF FIGURES

Fig. 3.1 System Workflow Diagram	15
Fig. 7.1. Portal Authentication And Registration Interfaces	43
Fig. 7.2. Hod (Admin) Central Management Dashboard	44
Fig. 7.3. Chemical Inventory Managed By Admin	45
Fig. 7.4. Apparatus Engine Management	46
Fig. 7.5. Staff Instructional Dashboard	47
Fig. 7.6. Experiment Initialization By Admin.....	47
Fig. 7.7. Experiment Initialization By Admin.....	48
Fig. 7.8. Database-Linked Chemical Assignment Modal	49
Fig. 7.9. Staff Instructional Dashboard	50
Fig. 7.10. Student Academic Progress Dashboard	50
Fig. 7.11.1 Interactive Apparatus Selection And Workspace Setup	51
Fig. 7.11.2 Titration Simulation In Progress	52
Fig. 7.11.3: Real-Time Procedural Marking And Penalty Assistant.....	53
Fig. 7.11.4 Final Authenticated Pdf Lab Appraisal Report.....	54